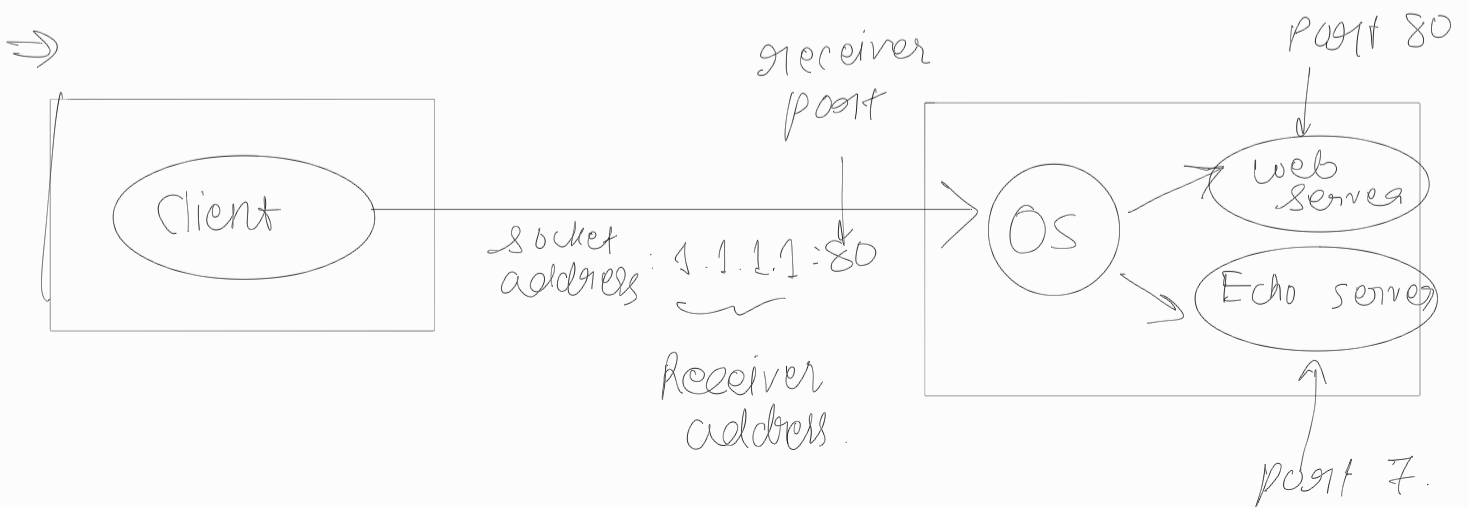
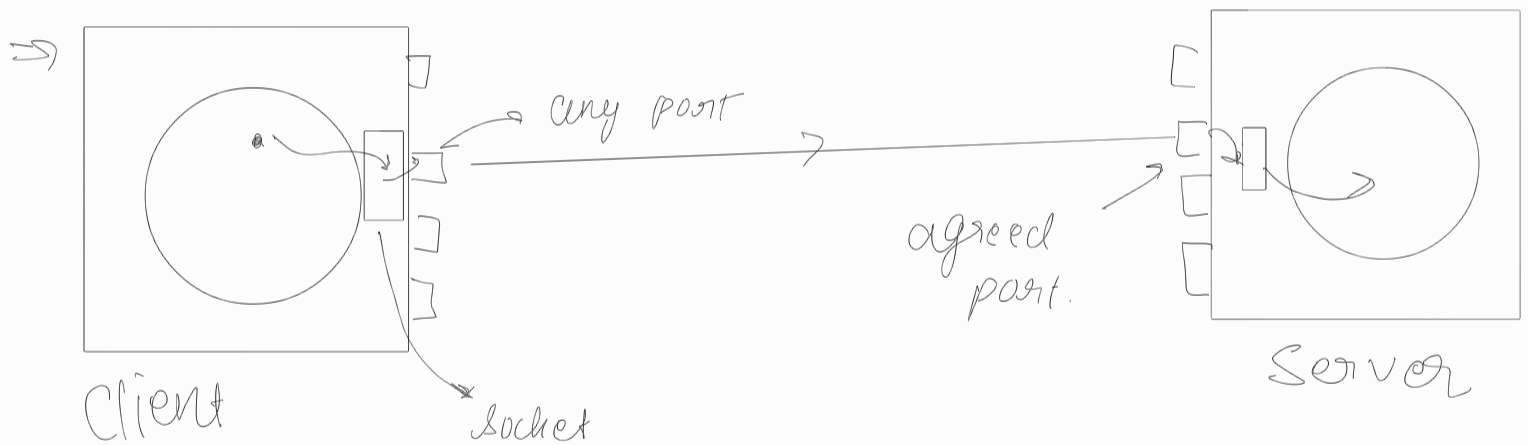


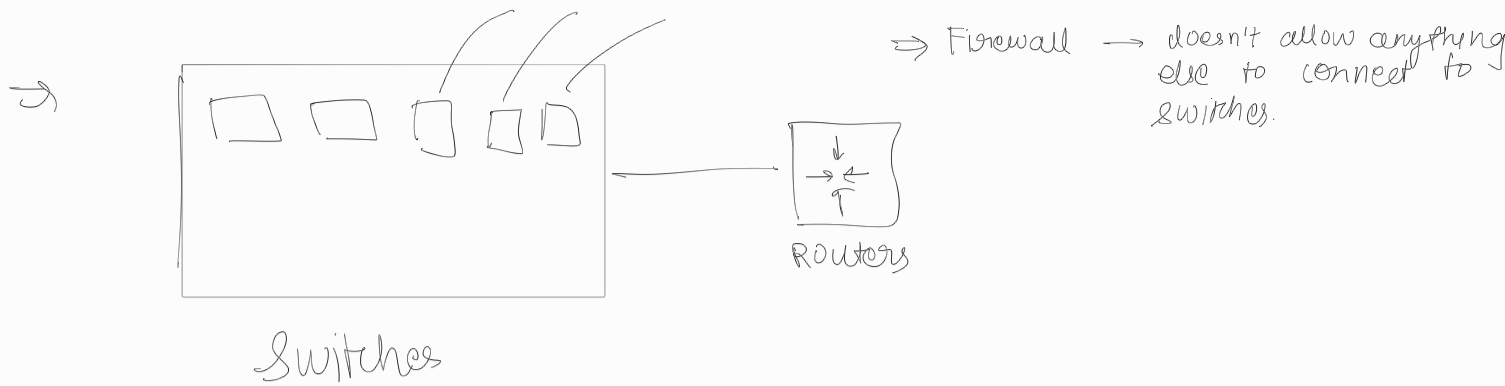
⇒ socket → endpoint of 2-way communication betⁿ 2 programs (on diff. machines)

⇒ IP address → address of a machine in a network.

⇒ ports → for process - process communication.

⇒ socket address → associated with IP and port address





⇒ Hubs v/s Switches

Hub: sends to everyone

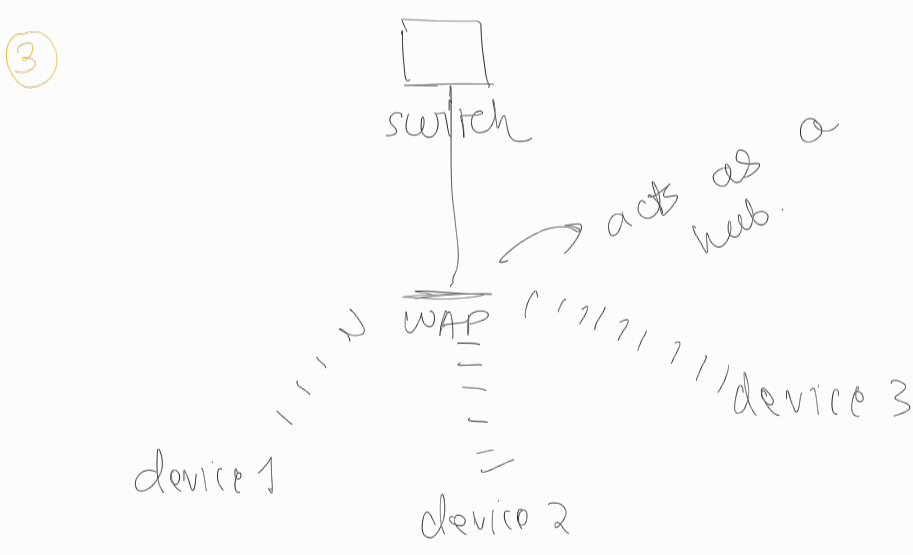
Switch: only to intended person.

⇒ Switch is Layer 2 device ⇒ learns MAC addresses of all devices on network. → learns source MAC address of a frame and stores in CAM Table.

Note:

- ① Layer 1 = electrical signals
- ② clients talk on Layer 3 (IP addresses) messages (receive/send) → called Packets
- ③ Layer 2 Messages = frames

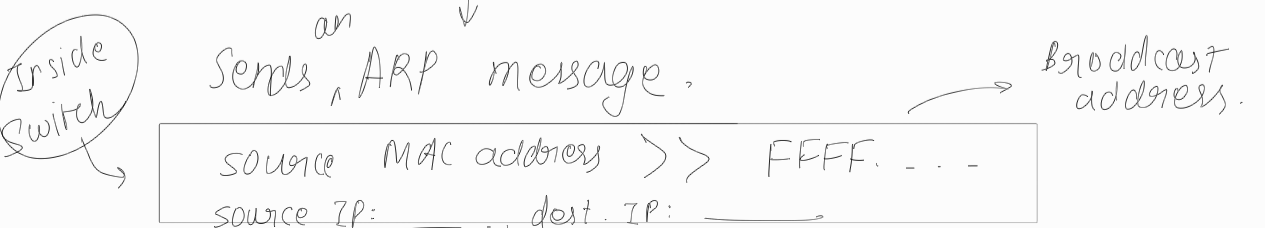
(content addressable memory)



④ CLI of Switch → "show mac-address-table"

⇒ sender must know receiver's MAC address

⇒ let's say sender knows IP address (receiver)



message sent to everyone (to find out whose)
↓
IP address is dest. IP

designated member responds with his
mac address to the switch. → source

⇒ why routers needed?

⇒ when dest. IP is on another network
(very diff. IP address), sender sends ARP
msg to find Router (Gateway IP address)

L3 = source
to
receiver

L2 =
source to
Router

↓
Sends msg to Router → Router is
↓
3 device

But, Router doesn't know
MAC Address of receiver

⇒ Receives packet

L2 = router
to
receiver

L3 = source
to
receiver

↓
Sends ARP to Switch 2 ⇒ ∴ MAC of receiver

∴ Johnny sends msg to receiver
after this process.

DNS??

TCP/IP :

- ⇒ Application
- ⇒ Transport
- ⇒ Network
- ⇒ Data link
- ⇒ Physical

OSI :

- ⇒ Application
- ⇒ Presentation
- ⇒ Session
- ⇒ Transport
- ⇒ Network
- ⇒ Data links
- ⇒ Physical

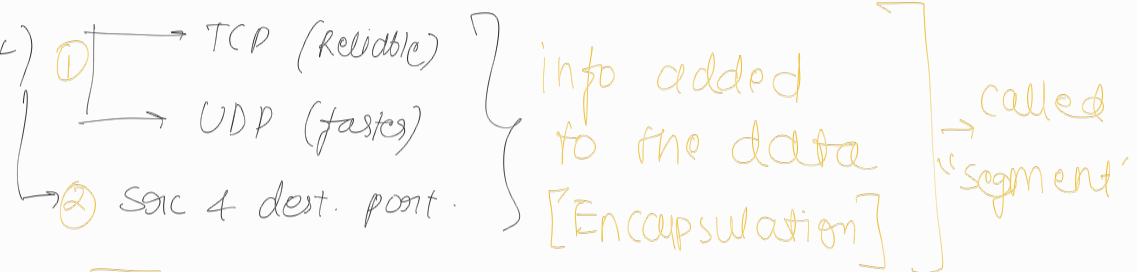
↑
advice,
People's
Sales
fake
not
do
Please

Note: WAP operates on layer 2. Hub → Layer 1.

↳ repeats electrical signals
(like repeater)

⇒ only http/https → secure

⇒ L7 → browser → http client sends
http request

⇒ L4 → (Transport) 

⇒ 

⇒ L3 → adds IP addresses (src, dest) to data

⇒ 

⇒ L2 → frame



Note: ① Switch → looks at frame → has all the layers

switch actually can see L1 & L2, but uses L2 data } but, the switch only looks at the head of frame (i.e. Mac address)

② Router → looks at packet → has all the above layers

- Network → logical/physical addressing
- Transport - error free data delivery

but only looks at L3 header

⇒ IP address :

⇒ Router gives IP address.

⇒ Eg: $192.168.1.204$ → IP Address
 $255.255.255.0$ → Subnet mask
 ↓
 Corresponding part of IP address
 doesn't change within a network.

Diagram: Arrows from the first three octets of the IP address (192, 168, 1) point to the word "octets". An arrow from the first three octets of the subnet mask (255, 255, 255) points to the word "Network". Brackets under the first three octets of both the IP address and the subnet mask are connected by a vertical line pointing down to the text below.

Note:
— . — . — . 0
— . — . — . 255
 ↓
 Broadcast address.

These 2 addresses can't be used

Ports:

⇒ logical connection to communicate betⁿ programs.
⇒ determines the program that is to be used.
⇒ Eg: 80, 443 → http / https
 21 → FTP
 25 → Email.

⇒ Port is a door to a room.
 ↓
 Same IP address.

⇒ Local IP address → can only be used if you are part of given LAN

⇒ Public IP address

⇒ Socket → communication endpoint. ⇒ Eg: Internet, Bluetooth, etc.

Python socket:

⇒ AF_INET, SOCK_STREAM → TCP
SOCK_DGRAM → UDP.

⇒ TCP ⇒ sequential, can detect packet loss,
⇒ connection based
⇒

⇒ UDP: Not sequential, no order.

⇒ Eg: Skype → call → UDP
call request → TCP

Socket is a
Py object which
handles the sending
& receiving of msgs
⇒ End point of comms
on a network.
Socket address.

⇒ socket.gethostbyname → "local IP address"

⇒ socket.gethostname → "Human readable
name"

⇒ server.bind → give socket a place in
network

⇒ server.bind((host, port))
 ↓ ↓
 IP port no.

⇒ TCP Handshake → SYN + ACK → client → Server (acknowledgment of sync)

① client → server (sends sequence no. of
messages)

② server → client (server sends " " " ")

⇒ server.accept() → activates after TCP handshake

⇒ returns communication socket
& address

↳ client ip, port specific
to the client
frees up server socket.

⇒ server.listen(n) → makes socket actively listen for connections.

"waiting room" { ↳ number of unaccepted connections
that the server OS will hold, before
rejecting clients' connection attempts.

AES: symmetric block cipher.

⇒ takes 128 bits and gives out 128 bits.

AES-GCM: symmetric, stream cipher.

⇒ Text is broken into 128 bit blocks. ★★

⇒ Separately, counter blocks are prepared: —

⇒ 128 bits = 96 + 32 bits

(NONCE) (counter)

⇒ counter is some number 'n' represented in 32 bits.

⇒ this counter increments by 1 in consecutive counter blocks.

Apply AES encryption ⇒ gives keystream (128 bits)
(using key k)

⇒ Keystream XOR with message blocks ⇒ gives cipher text

★★
⇒ if message is not multiple of 128 bits, then only required no. of bits from the start of the key stream are XOR.

Note: NONCE is number used once → same among msg blocks in a msg, but diff among diff msgs.

↳ unique, but not secret

⇒ NONCE + counter → public



counter block

↓ AES → uses key k (private)

keystream → (private)



cipher (public)

Hash key(H)
↳ private

Note: Symmetric encryption → Receiver and sender have same private key.
(fast)

Asymmetric encryption → Sender & Receiver know each other's encryption & decryption key.
(slow)
sender encrypts a message using receiver's encryption key and thus receiver can decrypt.

Hybrid → exchange keys using asymm.
↳ exchange msgs using symm.

RSA: ⇒ rsa.generate_private_key (public exponent key-size =
key.public_key()
key.private_key())

Serialisation

⇒ converting data structures into a stream of bits

⇒ PEM → used in cryptography → encodes underlying primes & maths into byte arrays, which can be verified globally.

⇒ pem private-key

= key.private_bytes(serialization.Encoding.PEM,
"PrivateFormat.PKCS8",
Serialization.NoEncryption())

→ wrapper (PEM file)

→ public key cryptography standard

⇒ pem-public key

= key.public_key.public_bytes

→ unencrypted file on server.

(encoding , format)

(No need to encrypt RSA key)

⇒ socket.sendall() → for PEM (reliable)

Note: socket.recv(bytes)

⇒ RSA-OAEP

Server: receives RSA-OAEP encrypted data, containing AES key.

⇒ OAEP adds noise to hide the fact that our secret key is 32 bytes

⇒ private_key.decrypt(message, padding, algorithm, mgf → mask-generation function)

privatekey.decrypt

encrypt()
decrypt()] → AES function (crypto.py)

→ Avoid

Note: use encode, decode with human readable text.

→ AES_CCM directly gives encrypted msg in bytes (same as above)

Phase 4: After setting up hybrid connection in phase 3, you authenticate:

- ⇒
- ① Client RSA key
 - ② Store RSA key in json
 - ③ server

Note: Hashing → used to check all bits of data are correctly received, i.e. even if a bit is changed, hash is changed.

should change even when bit changes

No 2 'does' should have same hash.
(Hash collisions)

fast

Eg: 5 - address → 4, 9, 10, 11 ⇒ hash = 34

⇒ Signer → Verifier
Document → Document

Private key

↑ Public

Signature → Signature

⇒ Signer → Verifier
Document → Document

↓
hash

↓ Private key

Signature
(Encrypted hash) → also, public key.

↓
hash

↑ Public key.

signature

must be same

JSON script we:

⇒ rsa_generate → Python object

↓ private - bytes, etc.
PEM bytes

↓ .decode('utf-8')
string
(stored in JSON)

client ↔ server

Note: socket.send()
must be in bytes
⇒ b"string"

Note: .encode() or .decode() should be used
only when bytes were originally
generated from text.

↳ use mapping like $1 \rightarrow A$
 $2 \rightarrow B$, etc.

⇒ if there were no.s > 26 , etc.,
it would crash

in this case
use `.hex()`
to get string